Отладка в браузере Chrome

Все современные браузеры и большинство других сред разработки поддерживают инструменты для отладки – специальный графический интерфейс, который сильно упрощает отладку. Он также позволяет по шагам отследить, что именно происходит в коде.

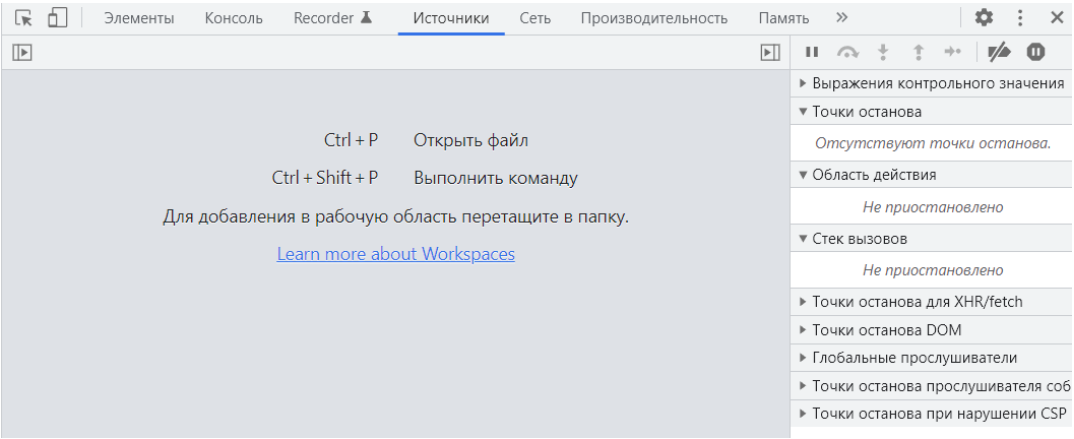
Мы будем использовать браузер Chrome, так как у него достаточно возможностей, в большинстве других браузеров процесс будет схожим.

Панель «Источники» («Sources»)

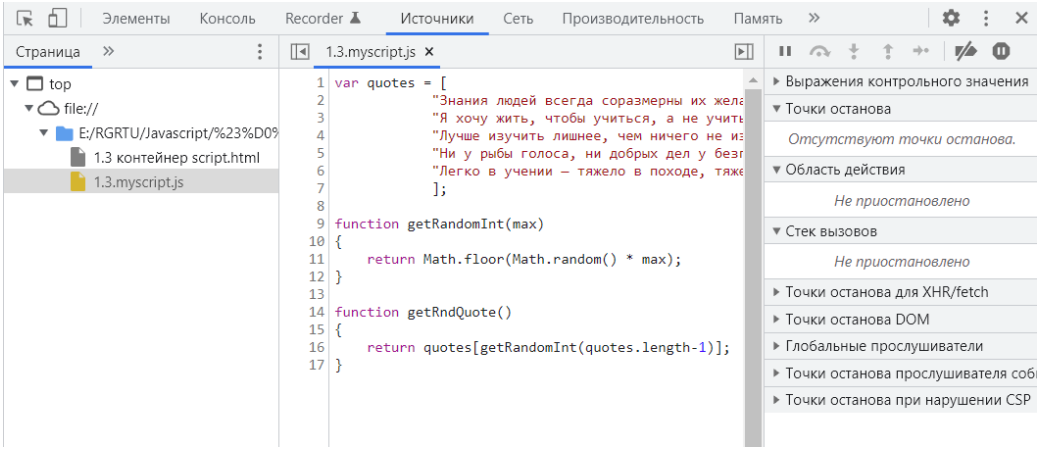
Разные версии Chrome могут выглядеть немного иначе, однако принципиальных отличий не будет.

- Работая в Chrome, откройте тестовую страницу.
- Включите инструменты разработчика, нажав F12.
- Щёлкните по вкладке «Источники» («sources»).

При первом запуске получаем следующее:



Далее щёлкните по кнопке чтобы открыть список файлов и выберите **myscript.js**:



Интерфейс состоит из трёх зон:

- В первой зоне показаны файлы HTML, JavaScript, CSS, включая изображения, используемые на странице. Здесь также могут быть файлы различных расширений Chrome.
- Вторая зона показывает исходный код.
- Третья зона отведена для инструментов отладки.

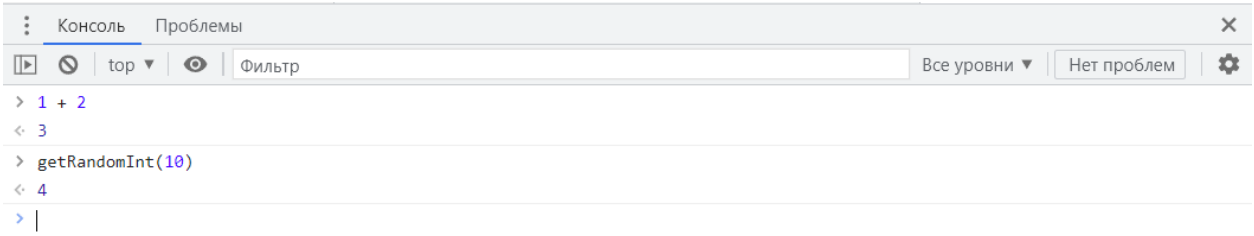
Чтобы скрыть список ресурсов и освободить экранное место для исходного кода, щёлкните по тому же переключателю.

Консоль

При нажатии на клавишу **Esc** в нижней части экрана вызывается консоль, где можно вводить команды и выполнять их клавишей **Enter**.

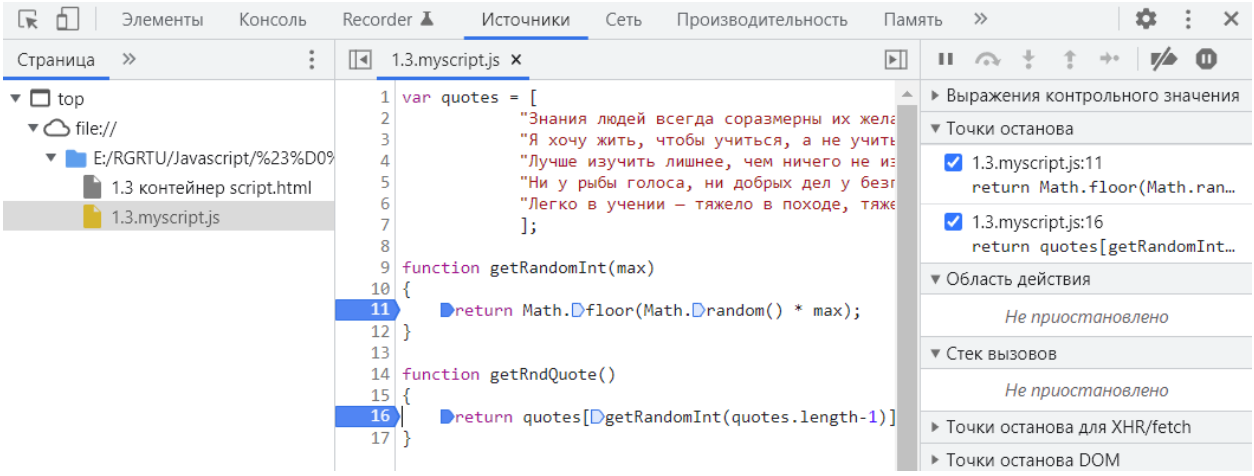
Результат выполнения инструкций сразу же отображается в консоли.

Например, результатом 1+2 будет 3, а инструкция getRandomInt(10) вызовет определённую нами в скрипте функцию генерации случайного числа:



Точки останова (breakpoints)

В файле **myscript.js** щёлкните по строчкам номер 11 и 16 (щёлкать надо по самой цифре). Номер строки будет окрашен в синий цвет.



Нами были установлены точки останова – участок кода, где отладчик автоматически приостановит исполнение JavaScript. Пока исполнение поставлено «на паузу», мы можем просмотреть текущие значения переменных, выполнить команды в консоли и т.д.

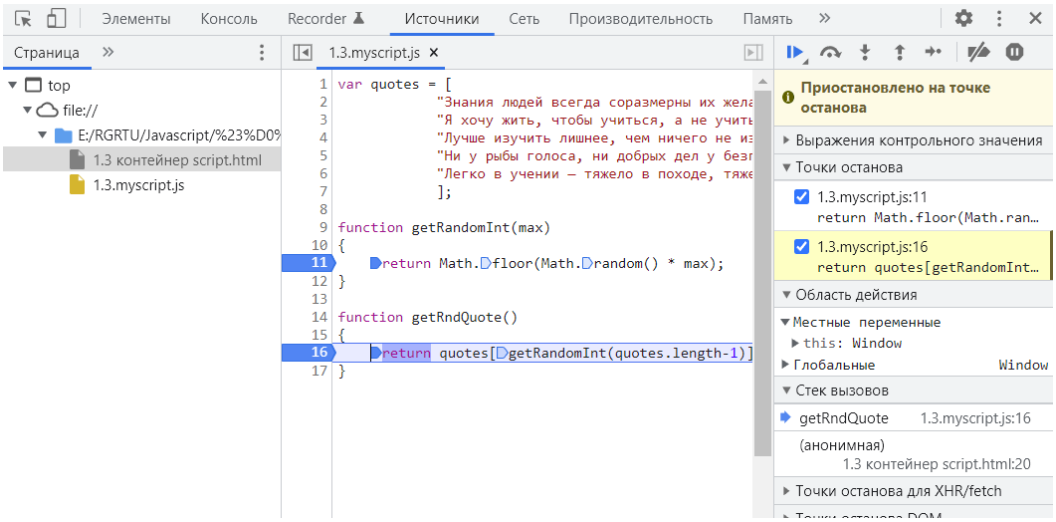
В правой части графического интерфейса находится список точек останова. Чтобы временно деактивировать точку – в общем списке снимите галочку напротив ненужной в данный момент точки. Удалить точку – щёлкните в списке по ней правой кнопкой мыши и выберите «Удалить» («Remove») или ещё раз щёлкните по номеру строки в коде.

Можно задать и условную точку останова – щёлкните правой кнопкой мыши по номеру строки в коде, выберите «Редактировать» («Edit»). Далее если задать выражение, то именно при его истинности выполнение кода будет приостановлено. Этот метод используется, когда выполнение кода нужно остановить при присвоении определённого выражения какой-либо переменной или при определённых параметрах функции.

Выполнение кода можно также приостановить с помощью команды **debugger;** прямо изнутри самого кода. Способ удобен тем, что можно продолжить работать в редакторе кода без необходимости переключения в браузер для выставления точки останова:

```
function getRandomInt(max)
{
    debugger; // <-- здесь выполнение прерывается
    return Math.floor(Math.random() * max);
}
```

В нашем примере функция getRndQuote() вызывается во время загрузки страницы, поэтому для начала отладки проще всего её перезагрузить - нажмите **F5**. Выполнение прервётся на 16 строчке:



Теперь в правой зоне у нас находится вся необходимая информация для отладки:

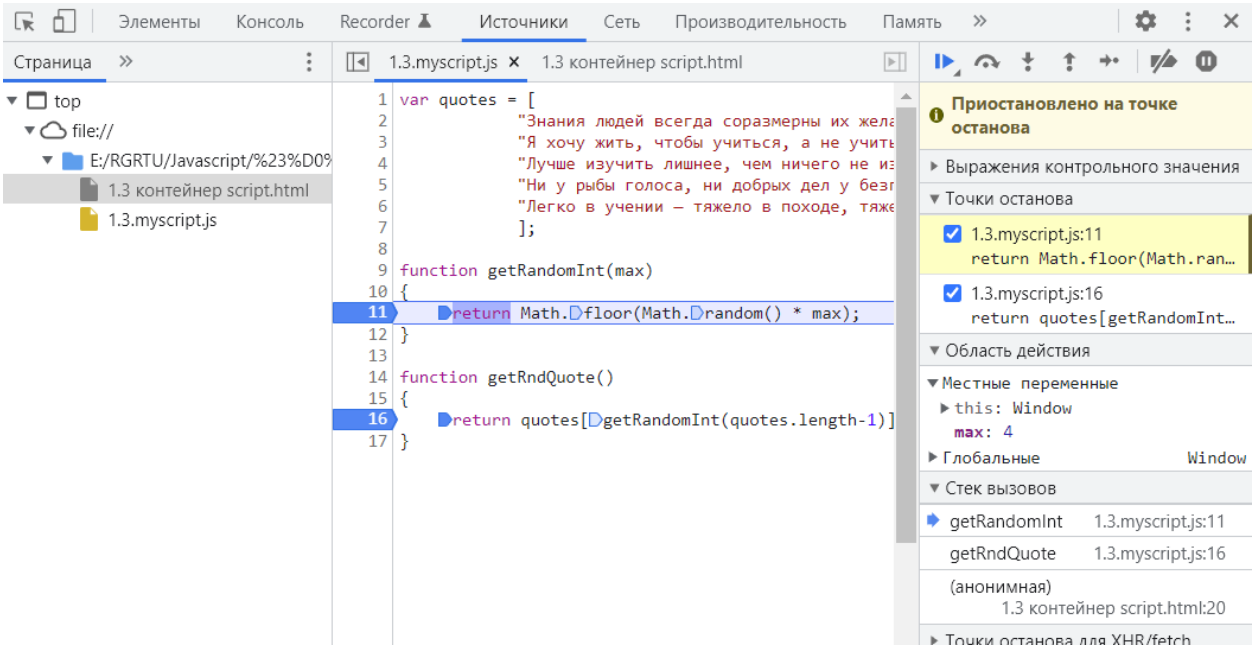
- «Выражения контрольного значения» («Watch») показывает текущие значения выражений. Нажмите на **+** и введите выражение. В процессе выполнения отладчик автоматически пересчитывает и выводит его значение.
- «Стек вызовов» («Call Stack») показывает последовательность вызовов функций. В нашем примере отладчик работает с функцией getRndQuote(), вызванной скриптом из файла html (там нет функции, поэтому вызов «анонимный»). При нажатии на элемент списка (например, на «анонимный») отладчик переходит к соответствующему коду, и нам представляется возможность его проанализировать.
- «Область действия» («Scope») показывает текущие локальные и глобальные переменные.

Пошаговое выполнение скрипта

В правой части панели для пошагового выполнения есть несколько кнопок. Рассмотрим их.

– продолжить выполнение. Быстрая клавиша – F8.

Возобновляет выполнение кода. Если больше нет точек останова, отладчик прекращает работу и позволяет приложению работать дальше. Вот, что мы увидим, кликнув на неё:



Выполнение кода возобновилось, дошло до другой точки останова внутри getRandomInt(), и отладчик снова приостановил выполнение. Обратите внимание стеке вызовов появился ещё один вызов. После этого можно сделать следующее:

– пропустить следующий вызов функции. Быстрая клавиша – F10.

– перейти к следующему вызову функции. Быстрая клавиша – F11. В отличие от предыдущей команды, здесь мы «заходим» во вложенные функции и шаг за шагом проходим по скрипту.

– выйти из текущей функции. Быстрая клавиша – Shift+F11. Выполнение кода остановится на самой последней строчке текущей функции. Этот метод применяется, когда мы случайно нажали и зашли в функцию, но нам она неинтересна и мы как можно скорее хотим из неё выйти.

– выполнить следующий шаг.

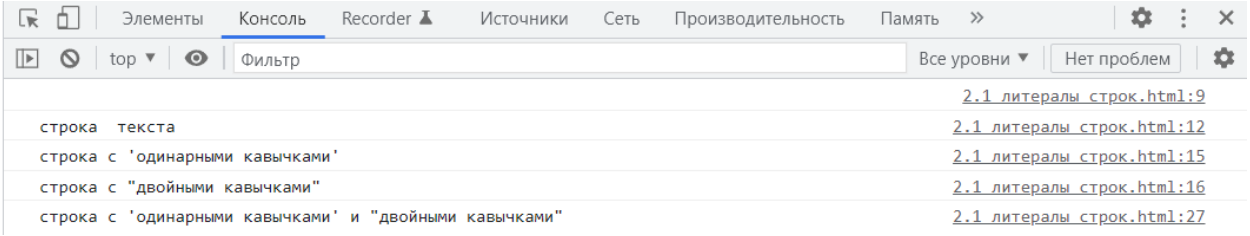
– активировать/деактивировать все точки останова. Эта кнопка не влияет на выполнение кода, она лишь позволяет массово включить/отключить точки останова.

– разрешить/запретить остановку выполнения в случае возникновения ошибки. Если опция включена и инструменты разработчика открыты, любая ошибка в скрипте приостанавливает выполнение кода, что позволяет его проанализировать. Поэтому если скрипт завершается с ошибкой, открываем отладчик, включаем эту опцию, перезагружаем страницу и локализуем проблему.

Если щёлкнуть правой кнопкой мыши по строчке кода, в контекстном меню можно выбрать опцию «продолжить до этого места» («Continue to here»). Этот метод используется, когда нам нужно продвинуться на несколько шагов вперёд до нужной строки, но лень выставлять точки останова.

Логирование

Если нужно что-то вывести в консоль из кода, применяется функция **console.log()**. Консоль можно открыть через инструменты разработчика – выберите вкладку «Консоль» или нажмите Esc, находясь в другой вкладке – консоль откроется в нижней части интерфейса.



Итого

Приостановить выполнение скрипта можно тремя способами:

- Точками останова.
- Использованием в коде команды debugger.
- При ошибке (если инструменты разработчика открыты и опция включена).

При остановке мы можем отлаживать – проанализировать переменные и пошагово пройти по процессу, что поможет отыскать проблему. Нами описаны далеко не все инструменты разработчика. С полным руководством можно ознакомиться здесь:

https://developers.google.com/web/tools/chrome-devtools

Для простой отладки вполне достаточно сведений из этого документа, но в дальнейшем рекомендуем вам изучить официальное руководство, если вы собираетесь разрабатывать для браузеров.